Checkpoint 2 — Gramática e Parser Recursivo Descendente

Parte 1 — Gramática final da linguagem

Maria Rita Vieira Souza- 12321BCC024, Ana Clara Morais de Carvalho- 12311BCC048

Link para o Github do projeto: https://github.com/mariarvsouza/GBC071-Construcao-de-Compiladores

1 Gramática da Linguagem

Em algumas produções foram necessárias alteraçeõs para remover a recursão à esquerda e prefixos comuns a mais de uma alternativa (fatoração à esquerda).

1.1 Fatoração à esquerda da declaração de variáveis e funções

A declaração de uma variável e a declaração de uma função começam ambas com o tipo seguido do identificador, além disso, as duas opções para a declaração de uma variável (”number i;” ou ”number i=0”) também compartilham o mesmo prefixo.

Antes da fatoração:

`Stmt -> Type "ID" "SEMICOLON" | Type "ID" "ASSIGN" Expr "SEMICOLON" | Type "ID" "LPAREN" ParamList "RPAREN" Block | Assign | IfStmt | ForStmt | ReturnStmt | BreakStmt | Block`

Como as três alternativas iniciam com o mesmo prefixo, é necessário usar a fatoração à esquerda.

Depois da fatoração:

`Stmt -> Type "ID" Decl' | Assign | IfStmt | ForStmt | ReturnStmt | BreakStmt | Block`

`Decl' -> "LPAREN" ParamList "RPAREN" Block | VarDecl'`

`VarDecl' -> "SEMICOLON" | "ASSIGN" Expr "SEMICOLON"`

1.2 Fatoração à esquerda para o "Dangling Else"(IfStmt)

A estrutura pode ter apenas o bloco if ou incluir o bloco else.

Antes da fatoração:

`IfStmt -> "KW_IF" "LPAREN" Expr "RPAREN" Block | "KW_IF" "LPAREN" Expr "RPAREN" Block "KW_ELSE" Block`

Como as duas alternativas iniciam com o mesmo prefixo, é necessário usar a fatoração à esquerda.

Depois da fatoração:

`IfStmt -> "KW_IF" "LPAREN" Expr "RPAREN" Block IfStmt'`

`IfStmt' -> "KW_ELSE" Block | ϵ`

1.3 Fatoração à esquerda para a inicialização do For (ForStmt e ForInit)

O laço for pode iniciar com uma atribuição simples ("i=0") ou com a declaração de uma variável ("int i=0").

Antes da fatoração:

`ForStmt -> "KW_FOR" "LPAREN" Type "ID" "ASSIGN" Expr "SEMICOLON" Expr "SEMICOLON" AssignBase "RPAREN" Block | "KW_FOR" "LPAREN" AssignBase "SEMICOLON" Expr "SEMICOLON" AssignBase "RPAREN" Block`

Como as duas alternativas iniciam com o mesmo prefixo, é necessário usar a fatoração à esquerda.

Depois da fatoração:

`ForStmt -> "KW_FOR" "LPAREN" ForInit "SEMICOLON" Expr "SEMICOLON" AssignBase "RPAREN" Block`

`ForInit -> Type "ID" "ASSIGN" Expr | AssignBase`

1.4 Fatoração à esquerda para o retorno (ReturnStmt)

Uma função pode retornar um valor (como "return x;") ou não retornar nada ("return;").

Antes da fatoração:

`ReturnStmt -> "KW_RETURN" Expr "SEMICOLON" | "KW_RETURN" "SEMICOLON"`

Como as duas alternativas iniciam com o mesmo prefixo, é necessário usar a fatoração

à esquerda.

Depois da fatoração:

`ReturnStmt -> "KW_RETURN" ReturnStmt'`

`ReturnStmt' -> Expr "SEMICOLON" | "SEMICOLON"`

1.5 Fatoração à esquerda para chamadas de função e identificadores (Atom)

Nas derivações de Atom, tanto identificadores simples ("ID"), como chamadas de função, iniciam com o token "ID".

Antes da fatoração:

`Atom -> "ID" | "ID" "LPAREN" ArgList "RPAREN" | "NUMBER_LIT" | "REAL_LIT" | "CHAR_LIT" | "STR_LIT" | "TRUE_LIT" | "FALSE_LIT" | "LPAREN" Expr "RPAREN"`

Como as duas alternativas iniciam com o mesmo prefixo, é necessário usar a fatoração à esquerda.

Depois da fatoração:

`Atom -> "ID" Atom' | "NUMBER_LIT" | "REAL_LIT" | "CHAR_LIT" | "STR_LIT" | "TRUE_LIT" | "FALSE_LIT" | "LPAREN" Expr "RPAREN"`

`Atom' -> "LPAREN" ArgList "RPAREN" |` ϵ

1.6 Remoção da recursão à esquerda do OU (OrExpr)

Antes da alteração:

`OrExpr -> OrExpr "OR" AndExpr | AndExpr`

Depois da alteração:

`OrExpr -> AndExpr OrExpr'`

`OrExpr' -> "OR" AndExpr OrExpr' |` ϵ

1.7 Remoção da recursão à esquerda do AND (AndExpr)

Antes da alteração:

`AndExpr -> AndExpr "AND" EqExpr | EqExpr`

Depois da alteração:

AndExpr -> EqExpr AndExpr'

AndExpr' -> "AND" EqExpr AndExpr' | ϵ

1.8 Remoção da recursão à esquerda da igualdade e diferença (EqExpr)

Antes da alteração:

EqExpr -> EqExpr "EQ" RelExpr | EqExpr "NE" RelExpr | RelExpr

Depois da alteração:

EqExpr -> RelExpr EqExpr'

EqExpr' -> "EQ" RelExpr EqExpr' | "NE" RelExpr EqExpr' | ϵ

1.9 Remoção da recursão à esquerda dos operadores relacionais (RelExpr)

Antes da alteração:

RelExpr -> RelExpr "GT" AddExpr | RelExpr "GE" AddExpr | RelExpr "LT" AddExpr | RelExpr "LE" AddExpr | AddExpr

Depois da alteração:

RelExpr -> AddExpr RelExpr'

RelExpr' -> "GT" AddExpr RelExpr' | "GE" AddExpr RelExpr' | "LT" AddExpr RelExpr' | "LE" AddExpr RelExpr' | ϵ

1.10 Remoção da recursão à esquerda da adição e subtração (AddExpr)

Antes da alteração:

AddExpr -> AddExpr "PLUS" MultExpr | AddExpr "MINUS" MultExpr | MultExpr

Depois da alteração:

AddExpr -> MultExpr AddExpr'

AddExpr' -> "PLUS" MultExpr AddExpr' | "MINUS" AddExpr AddExpr' | ϵ

1.11 Remoção da recursão à esquerda da multiplicação, divisão e resto (MultExpr)

Antes da alteração:

MultExpr -> MultExpr "MULT" UnaryExpr | MultExpr "DIV" UnaryExpr | MultExpr "MOD" UnaryExpr | UnaryExpr

Depois da alteração:

MultExpr -> UnaryExpr MultExpr'

MultExpr' -> "MULT" UnaryExpr MultExpr' | "DIV" UnaryExpr MultExpr' | "MOD" UnaryExpr MultExpr' | ϵ

1.12 Gramática final:

Gramática final na notação BNF:

```
Programa -> StmtList
StmtList -> Stmt StmtList | ϵ
Stmt -> Type "ID" Decl' | Assign | IfStmt | ForStmt | ReturnStmt | BreakStmt |
Block
Block-> "LBRACE" StmtList "RBRACE"

Decl' -> "LPAREN" ParamList "RPAREN" Block | VarDecl'
VarDecl' -> "SEMICOLON" | "ASSIGN" Expr "SEMICOLON"

Param -> Type "ID"
ParamList -> Param ParamList' | ϵ
ParamList' -> "COMMA" Param ParamList' | ϵ

ArgList -> Expr ArgList' | ϵ
ArgList' -> "COMMA" Expr ArgList' | ϵ
AssignBase -> "ID" "ASSIGN" Expr
Assign -> AssignBase "SEMICOLON"

IfStmt -> "KW_IF" "LPAREN" Expr "RPAREN" Block IfStmt'
IfStmt' -> "KW_ELSE" Block | ϵ

ForStmt -> "KW_FOR" "LPAREN" ForInit "SEMICOLON" Expr "SEMICOLON" AssignBase
"RPAREN" Block
ForInit -> Type "ID" "ASSIGN" Expr | AssignBase

ReturnStmt -> "KW_RETURN" ReturnStmt'
ReturnStmt' -> Expr "SEMICOLON" | "SEMICOLON"
BreakStmt -> "KW_BREAK" "SEMICOLON"

Type -> "KW_NUMBER" | "KW_REAL" | "KW_CHAR" | "KW_STR" | "KW_BOOL" | "KW_VOID"
```

```
Expr -> OrExpr
OrExpr -> AndExpr OrExpr'
OrExpr' -> "OR" AndExpr OrExpr' | ϵ
AndExpr -> EqExpr AndExpr'
AndExpr' -> "AND" EqExpr AndExpr' | ϵ
EqExpr -> RelExpr EqExpr'
EqExpr' -> "EQ" RelExpr EqExpr' | "NE" RelExpr EqExpr' | ϵ
RelExpr -> AddExpr RelExpr'
RelExpr' -> "GT" AddExpr RelExpr' | "GE" AddExpr RelExpr' | "LT" AddExpr RelExpr'
| "LE" AddExpr RelExpr' | ϵ
AddExpr -> MultExpr AddExpr'
AddExpr' -> "PLUS" MultExpr AddExpr' | "MINUS" MultExpr AddExpr' | ϵ
MultExpr -> UnaryExpr MultExpr'
MultExpr' -> "MULT" UnaryExpr MultExpr' | "DIV" UnaryExpr MultExpr' | "MOD"
UnaryExpr MultExpr' | ϵ
UnaryExpr -> "NOT" UnaryExpr | Atom
Atom -> "ID" Atom' | "NUMBER_LIT" | "REAL_LIT" | "CHAR_LIT" | "STR_LIT" | "TRUE_LIT"
| "FALSE_LIT" | "LPAREN" Expr "RPAREN"
Atom' -> "LPAREN" ArgList "RPAREN" | ϵ
```

2 FIRST E FOLLOW

Não-terminal	**FIRST**	**FOLLOW**
Programa	KW_NUMBER, KW_REAL, KW_CHAR, KW_STR, KW_BOOL, KW_VOID, ID, KW_IF, KW_FOR, KW_RETURN, KW_BREAK, LBRACE, ϵ	$
StmtList	KW_NUMBER, KW_REAL, KW_CHAR, KW_STR, KW_BOOL, KW_VOID, ID, KW_IF, KW_FOR, KW_RETURN, KW_BREAK, LBRACE, ϵ	$, RBRACE
Stmt	KW_NUMBER, KW_REAL, KW_CHAR, KW_STR, KW_BOOL, KW_VOID, ID, KW_IF, KW_FOR, KW_RETURN, KW_BREAK, LBRACE	KW_NUMBER, KW_REAL, KW_CHAR, KW_STR, KW_BOOL, KW_VOID, ID, KW_IF, KW_FOR, KW_RETURN, KW_BREAK, LBRACE, RBRACE, $
Block	LBRACE	KW_NUMBER, KW_REAL, KW_CHAR, KW_STR, KW_BOOL, KW_VOID, ID, KW_IF, KW_FOR, KW_RETURN, KW_BREAK, LBRACE, RBRACE, $, KW_ELSE

Não-terminal	**FIRST**	**FOLLOW**
Decl′	LPAREN, SEMICOLON, ASSIGN	KW_NUMBER, KW_REAL, KW_CHAR, KW_STR, KW_BOOL, KW_VOID, ID, KW_IF, KW_FOR, KW_RETURN, KW_BREAK, LBRACE, RBRACE, $
VarDecl′	SEMICOLON, ASSIGN	KW_NUMBER, KW_REAL, KW_CHAR, KW_STR, KW_BOOL, KW_VOID, ID, KW_IF, KW_FOR, KW_RETURN, KW_BREAK, LBRACE, RBRACE, $
Param	KW_NUMBER, KW_REAL, KW_CHAR, KW_STR, KW_BOOL, KW_VOID	COMMA, RPAREN
ParamList	KW_NUMBER, KW_REAL, KW_CHAR, KW_STR, KW_BOOL, KW_VOID, ϵ	RPAREN
ParamList′	COMMA, ϵ	RPAREN
ArgList	NOT, ID, NUMBER_LIT, REAL_LIT, CHAR_LIT, STR_LIT, TRUE_LIT, FALSE_LIT, LPAREN, ϵ	RPAREN
ArgList′	COMMA, ϵ	RPAREN
AssignBase	ID	SEMICOLON, RPAREN
Assign	ID	KW_NUMBER, KW_REAL, KW_CHAR, KW_STR, KW_BOOL, KW_VOID, ID, KW_IF, KW_FOR, KW_RETURN, KW_BREAK, LBRACE, RBRACE, $
IfStmt	KW_IF	KW_NUMBER, KW_REAL, KW_CHAR, KW_STR, KW_BOOL, KW_VOID, ID, KW_IF, KW_FOR, KW_RETURN, KW_BREAK, LBRACE, RBRACE, $
IfStmt′	KW_ELSE, ϵ	KW_NUMBER, KW_REAL, KW_CHAR, KW_STR, KW_BOOL, KW_VOID, ID, KW_IF, KW_FOR, KW_RETURN, KW_BREAK, LBRACE, RBRACE, $
ForStmt	KW_FOR	KW_NUMBER, KW_REAL, KW_CHAR, KW_STR, KW_BOOL, KW_VOID, ID, KW_IF, KW_FOR, KW_RETURN, KW_BREAK, LBRACE, RBRACE, $
ForInit	KW_NUMBER, KW_REAL, KW_CHAR, KW_STR, KW_BOOL, KW_VOID, ID	SEMICOLON

Não-terminal	**FIRST**	**FOLLOW**
ReturnStmt	KW_RETURN	KW_NUMBER, KW_REAL, KW_CHAR, KW_STR, KW_BOOL, KW_VOID, ID, KW_IF, KW_FOR, KW_RETURN, KW_BREAK, LBRACE, RBRACE, $
ReturnStmt′	NOT, ID, NUMBER_LIT, REAL_LIT, CHAR_LIT, STR_LIT, TRUE_LIT, FALSE_LIT, LPAREN, SEMICOLON	KW_NUMBER, KW_REAL, KW_CHAR, KW_STR, KW_BOOL, KW_VOID, ID, KW_IF, KW_FOR, KW_RETURN, KW_BREAK, LBRACE, RBRACE, $
BreakStmt	KW_BREAK	KW_NUMBER, KW_REAL, KW_CHAR, KW_STR, KW_BOOL, KW_VOID, ID, KW_IF, KW_FOR, KW_RETURN, KW_BREAK, LBRACE, RBRACE, $
Type	KW_NUMBER, KW_REAL, KW_CHAR, KW_STR, KW_BOOL, KW_VOID	ID
Expr	NOT, ID, NUMBER_LIT, REAL_LIT, CHAR_LIT, STR_LIT, TRUE_LIT, FALSE_LIT, LPAREN	SEMICOLON, RPAREN, COMMA
OrExpr	NOT, ID, NUMBER_LIT, REAL_LIT, CHAR_LIT, STR_LIT, TRUE_LIT, FALSE_LIT, LPAREN	SEMICOLON, RPAREN, COMMA
OrExpr′	OR, ϵ	SEMICOLON, RPAREN, COMMA
AndExpr	NOT, ID, NUMBER_LIT, REAL_LIT, CHAR_LIT, STR_LIT, TRUE_LIT, FALSE_LIT, LPAREN	OR, SEMICOLON, RPAREN, COMMA
AndExpr′	AND, ϵ	OR, SEMICOLON, RPAREN, COMMA
EqExpr	NOT, ID, NUMBER_LIT, REAL_LIT, CHAR_LIT, STR_LIT, TRUE_LIT, FALSE_LIT, LPAREN	AND, OR, SEMICOLON, RPAREN, COMMA
EqExpr′	EQ, NE, ϵ	AND, OR, SEMICOLON, RPAREN, COMMA
RelExpr	NOT, ID, NUMBER_LIT, REAL_LIT, CHAR_LIT, STR_LIT, TRUE_LIT, FALSE_LIT, LPAREN	EQ, NE, AND, OR, SEMICOLON, RPAREN, COMMA
RelExpr′	GT, GE, LT, LE, ϵ	EQ, NE, AND, OR, SEMICOLON, RPAREN, COMMA
AddExpr	NOT, ID, NUMBER_LIT, REAL_LIT, CHAR_LIT, STR_LIT, TRUE_LIT, FALSE_LIT, LPAREN	GT, GE, LT, LE, EQ, NE, AND, OR, SEMICOLON, RPAREN, COMMA
AddExpr′	PLUS, MINUS, ϵ	GT, GE, LT, LE, EQ, NE, AND, OR, SEMICOLON, RPAREN, COMMA

Não-terminal	**FIRST**	**FOLLOW**
MultExpr	NOT, ID, NUMBER_LIT, REAL_LIT, CHAR_LIT, STR_LIT, TRUE_LIT, FALSE_LIT, LPAREN	PLUS, MINUS, GT, GE, LT, LE, EQ, NE, AND, OR, SEMICOLON, RPAREN, COMMA
MultExpr′	MULT, DIV, MOD, ϵ	PLUS, MINUS, GT, GE, LT, LE, EQ, NE, AND, OR, SEMICOLON, RPAREN, COMMA
UnaryExpr	NOT, ID, NUMBER_LIT, REAL_LIT, CHAR_LIT, STR_LIT, TRUE_LIT, FALSE_LIT, LPAREN	MULT, DIV, MOD, PLUS, MINUS, GT, GE, LT, LE, EQ, NE, AND, OR, SEMICOLON, RPAREN, COMMA
Atom	ID, NUMBER_LIT, REAL_LIT, CHAR_LIT, STR_LIT, TRUE_LIT, FALSE_LIT, LPAREN	MULT, DIV, MOD, PLUS, MINUS, GT, GE, LT, LE, EQ, NE, AND, OR, SEMICOLON, RPAREN, COMMA
Atom′	LPAREN, ϵ	MULT, DIV, MOD, PLUS, MINUS, GT, GE, LT, LE, EQ, NE, AND, OR, SEMICOLON, RPAREN, COMMA

3 Condição LL(1)

Na gramática criada, todos os não-terminais seguem as regras: (a) não-terminais com múltiplas alternativas possuem FISRT disjuntos entre si, (b) no máximo uma alternativa deriva ϵ, (c) quando há uma alternativa anulável, o FISRT das demais é disjunto do FOLLOW do não-terminal. Logo, a gramática segue a condição LL(1), já que não encontramos nenhum não-terminal que viola alguma cláusula.